CS391 Web Application Development Project (1)

Assignment Date : 25.03.2026 09.00 AM
Deadline : 25.04.2026 11.59 PM (NO EXTENSION)

Contents

1. Project Description 2
2. Data Requirements 3
3. Technical Requirements 3
4. Main Page (index.html) 4
5. Search Results Page (searchResults.html) 5
6. Hotel Details Page (hotelDetails.html) 5
7. Payment Page (payment.html) 6
8. Screen Mockups 7
   a. index.html 7
      Figure 1: index.html: When user opens the index.html first time 8
      Figure 2: index.html: After search 8
   b. searchResults.html 9
      Figure 3: searchResults.html 9
   c. hotelDetails.html 10
      Figure 4: when user opens the hotelDetails.html page first time 10
      Figure 5: when user clicks “Book now” button 11
   d. payment.html 12
      Figure 6: Payment.html 12
9. RUBRIC 13

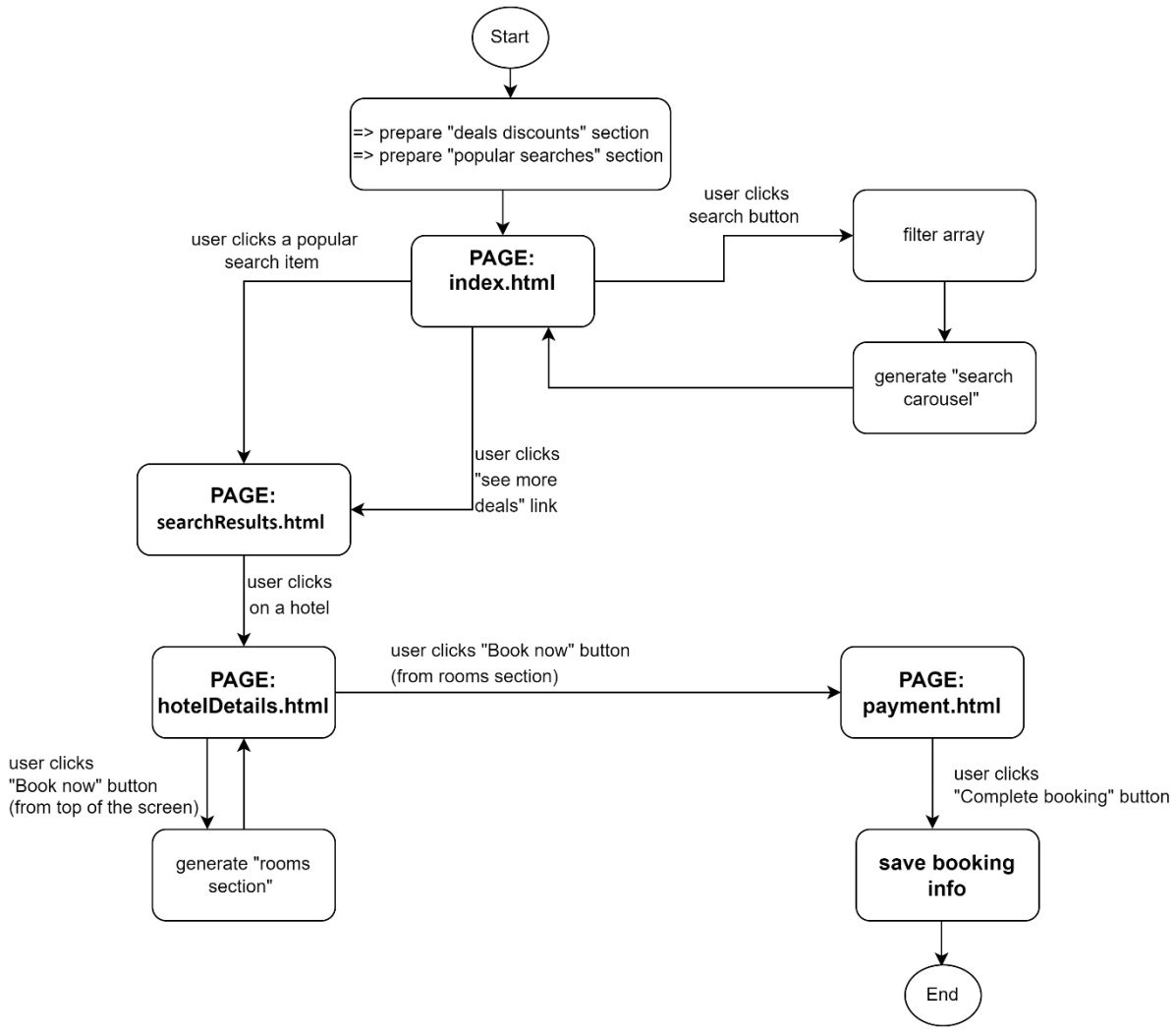
1. Project Description

For this project you will develop a simple hotel reservation application.

You need to create 4 **HTML** pages for this project:

1. index.html
2. searchResults.html
3. hotelDetails.html
4. payment.html

Do **not** create any HTML pages other than the ones listed above. Screen flow and page details are described below:



2. Data Requirements

You need different data sources for this project. You will use JavaScript arrays to meet the data requirements. You can use public data arrays or create your own arrays.

You need three different arrays:

1. Hotel list
2. Deals and discounts list
3. Popular searches list

Some mock data examples and links are provided below. If you generate your own arrays, you can use dummy text, but do not use the same values for each object. Your values must be sufficient to test your application.

Important:

1. Do not store images in your local environment.
2. You can create a folder in your project directory to store images if you want, but I recommend using **free internet-based images** to keep your application size small.
3. Your images must be **publicly accessible**.

Useful links:

You can use the **Mockaroo** application to generate mock JSON data:
https://www.mockaroo.com/

Other mock data websites:

- https://gist.github.com/Thaer-Sarakbi/e92e818f0a2c9d5467cf68e8c8833d65
- https://github.com/Lemoncode/simple-hotels-mock-rest-api
- https://github.com/WillGardella/hotels/tree/master

I have already downloaded some of these files. You can find the related files in the LMS: hotels-data.json, hotels1.json, hotels2.json, MOCK_DATA.json.

3. Technical Requirements

1. Use SessionStorage as much as possible. Do not use localStorage and global scope if is not really required.
2. Keep sessionStorage as much clean as possible.
3. Do not use any JavaScript framework or library (except Bootstrap). Use only pure JavaScript.
4. Do not use inline or embedded JavaScript and CSS.
5. Use Bootstrap for Responsive web design. You can also use custom CSS.
6. You can use Bootstrap components and JavaScript libraries.
7. Do not use any Restful services. Store and get your data from SessionStorage

4. Main Page (index.html)

The main page is the starting point of your application. This page **must be responsive** and contain **3 sections**: Search Bar, Deals and Discounts and Populer Searches

See **Figure 1** below for screen design.

1. **Search Bar:** This bar will be used for searching available hotels. It must contain 4 fields and all of them are required.
    a. Search text field
    b. Check-in and check-out date fields
    c. Guest and room count selection fields
2. **Deals and Discounts Section:**
    a. Display at least **4 deals or discounts**.
    b. Create an array to store the discounts and deals. Each object in the array must contain **title**, **details**, and **image URL** fields.
    c. Generate the content of this section dynamically by iterating over your array.
    d. You can store your images in your project directory or use images from the internet.
    e. The "Learn more" link does not need to redirect to another page.
3. **Popular Searches Section:**
    a. Create an array to store popular search data. Each object in the array must contain **search title**, some **details info** (average price, number of hotels, max/min prices, etc.), and **image URL**.
    b. Generate the content of this section dynamically by iterating over your array.
    c. Use the **Bootstrap carousel component** and display at least **4 items**. You can show **one slide or multiple slides** on the screen at a time.
    d. You can store your images in your project directory or use images from the internet.
    e. When a user clicks on a search item, `searchResults.html` must be displayed with the results for the selected search item. For example, if a user clicks on "Antalya," `searchResults.html` must show the hotels in Antalya. See **Figure 3** below**.**

Search function requirements

a. When a user clicks the **Search** button,
    i. filter your hotels array and find the related hotels. Use only the **text** entered in the search field. You cannot use dates, guest counts, or room data as search criteria, because your hotels array probably contains only hotel information, not reservation data.
        1. Your search function must check the user-entered search text in the **hotel title**, **description**, or **information fields**. Depending on your hotel data, you can also check other fields.
        2. Do not iterate over the array manually; use the **array.filter** method**.**
    ii. Remove the content of the Deals and Discounts section and display the search results in a **Bootstrap Carousel** component. Display **5 slides at a time** and sort the hotels by **hotel name in ascending order**.
    iii. Display a **"See more deals"** link at the top of the carousel.

iv. See **Figure 2** below for screen design

b. When a user clicks "**See more deals"** link, `searchResults.html` must be displayed with the filtered hotels (sorted by hotel name in ascending order).

5. Search Results Page (searchResults.html)

1. Must contain two parts:
   - Search Bar
   - Search Results
2. See Figure-3 below for screen design
3. The presentation of the search bar must be the same as in `index.html`.
4. Page must be responsive.
5. Display at least 4 hotels per row.
6. Each hotel card should contain an image and the hotel name. You can also display additional information about the hotel, such as location, city, star count, rating count, and average rating.
7. When the user makes another search, do not display `index.html` again; regenerate the page with the latest search results.
8. Hotels must be sorted by name in ascending order.
9. When the user clicks on a hotel, the `hotelDetails.html` page must be displayed with the related hotel information. You can store the selected hotel's data in `sessionStorage`.

6. Hotel Details Page (hotelDetails.html)

1. You can get the selected hotel's data, which was selected from the Search Results page, from `sessionStorage`.
2. The page must contain **three sections**:
   a. Hotel photos carousel
   b. Hotel information and rules
   c. Room selection
3. The page must be responsive.
4. See **Figure 4** below for the screen design.
5. Initially, the **Room Selection** section must be hidden.
6. When the user clicks the `Book Now` button, generate the Room Selection section dynamically. See Figure 5 below for the screen design.
7. You are free to modify the design of the Room Selection section.
8. When the user clicks the `Book Now` button in the Room Selection section, show the `payment.html` page. In this case, the room count and room type must be selected.
9. Store the hotel and room information in `sessionStorage` before displaying `payment.html`. Do not create a new object in `sessionStorage`; update the object that was created on the `SearchResults` page.

7. Payment Page (payment.html)

1. Get the selected hotel's reservation data from sessionStorage.
2. All form fields are required. Add any additional validation rules as needed. Check the data before submitting the form and display error messages if there are any validation errors.
3. You may optionally add additional fields for other guests' information, but this is not required.
4. When the user clicks the "`Complete the Booking`" button, create a reservation object and save it to `sessionStorage`. Clear all fields on the screen and show an alert box with a success message. Do not clear sessionStorage (I will check it)
5. Your reservation object must contain:
    - Hotel name and address
    - Reservation data (room count, room type, dates)
    - Guest data
    - Payment information
6. A sample Reservation object structure is privided below:

```
Reservation: {
 "hotel": {
  "name": "Antalya Hotel",
  "address": "123 Beach Road, Antalya"
 },
 "reservationData": {
  "roomCount": 2,
  "guestCount":4,
  "roomType": "Deluxe",
  "checkInDate": "2026-05-01",
  "checkOutDate": "2026-05-05"
 },
 "guestData": {
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "phone": "+90 555 123 4567"
 },
 "paymentInformation": {
  "cardInfo": {
   "cardNumber": "1234-5678-9012-3456",
   "cardHolder": "John Doe",
   "expiryDate": "05/28",
   "cvv": "123"
  },
  "paymentMethod": "Credit Card",
```

```
    "totalAmount": 500
  }
}
```

8. Screen Mockups

These mockup screens are provided for reference only. You may change the design as long as the required functions and specifications are met

a. index.html

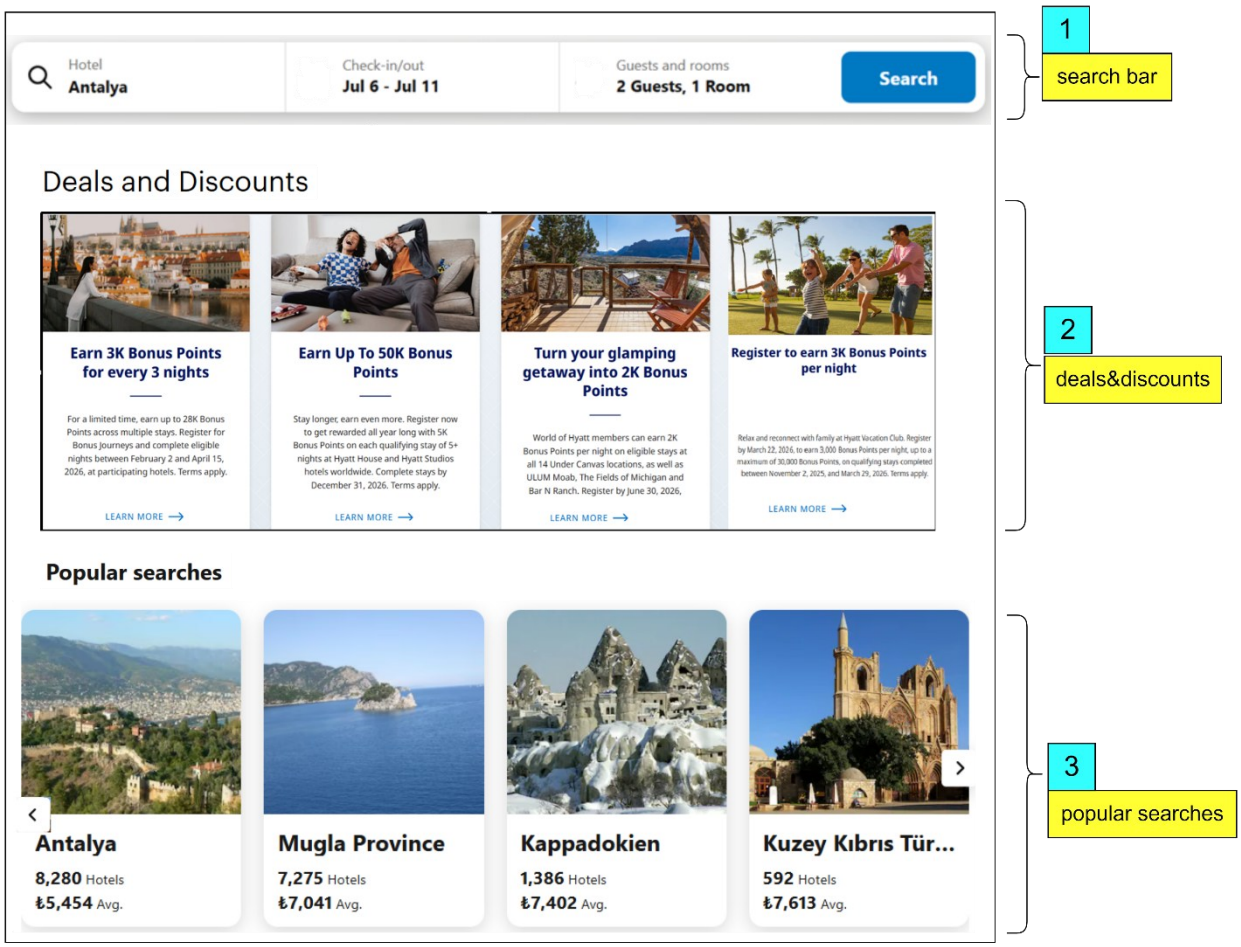


Figure 1: index.html: When user opens the index.html first time

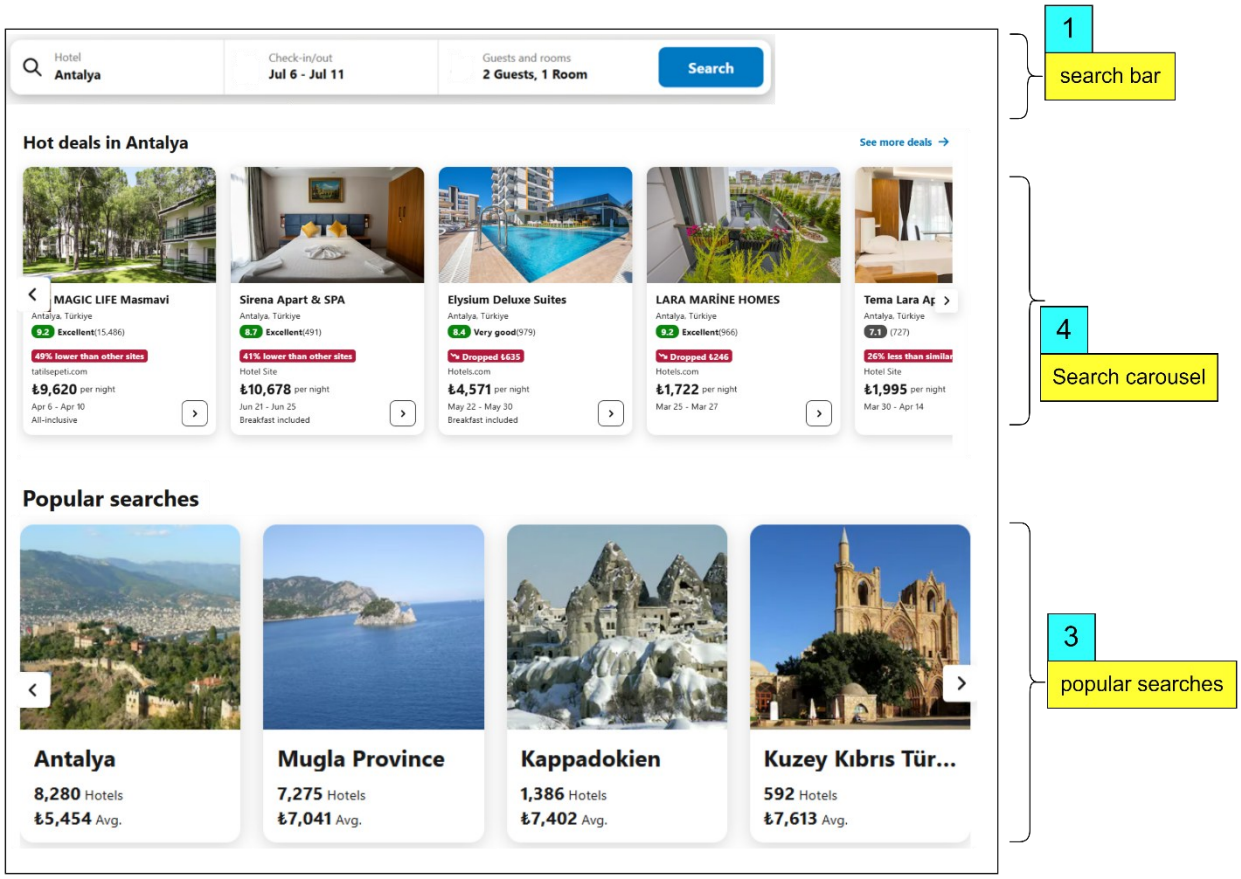


Figure 2: index.html: After search

b. searchResults.html

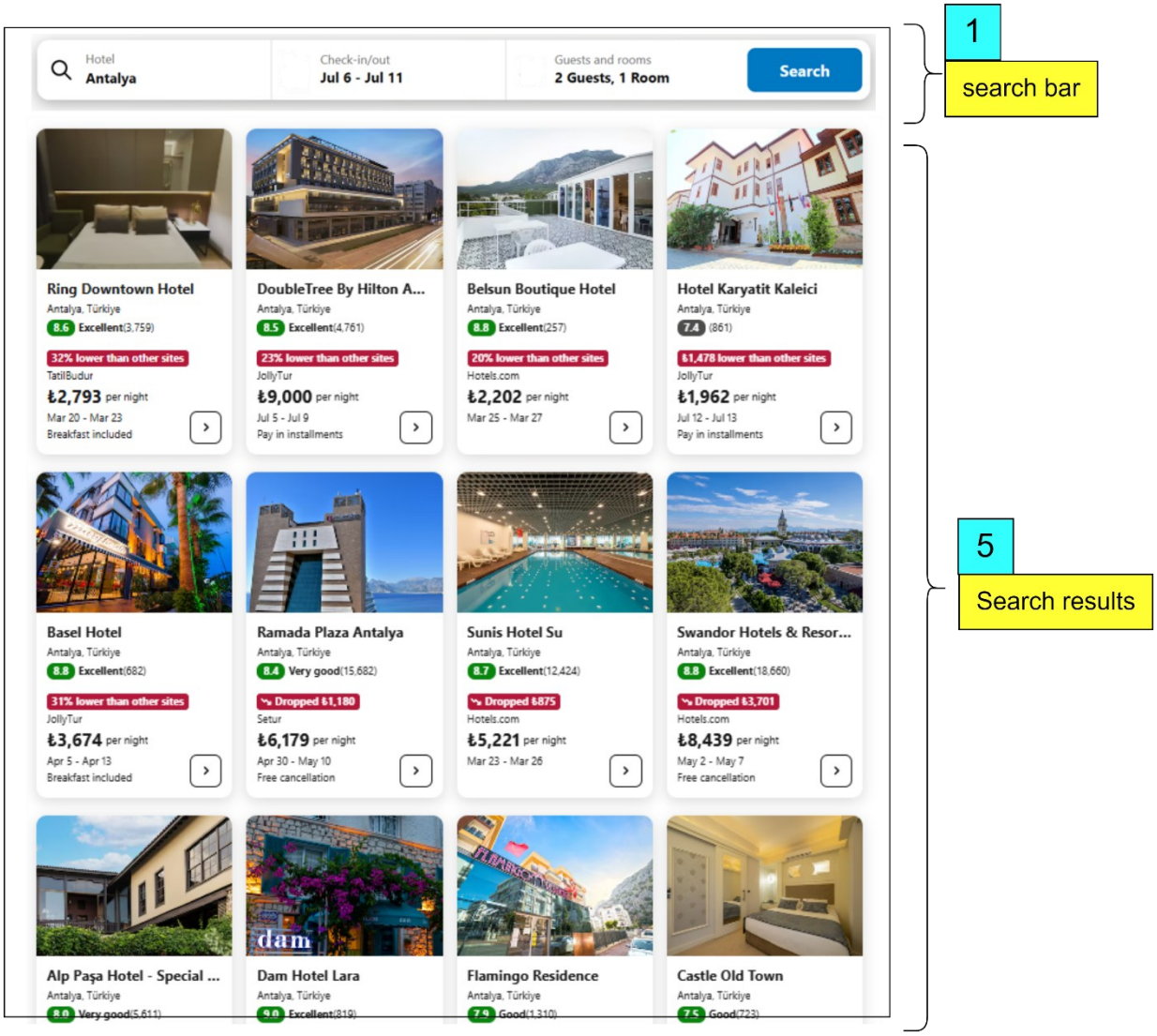


Figure 3: searchResults.html

c. hotelDetails.html

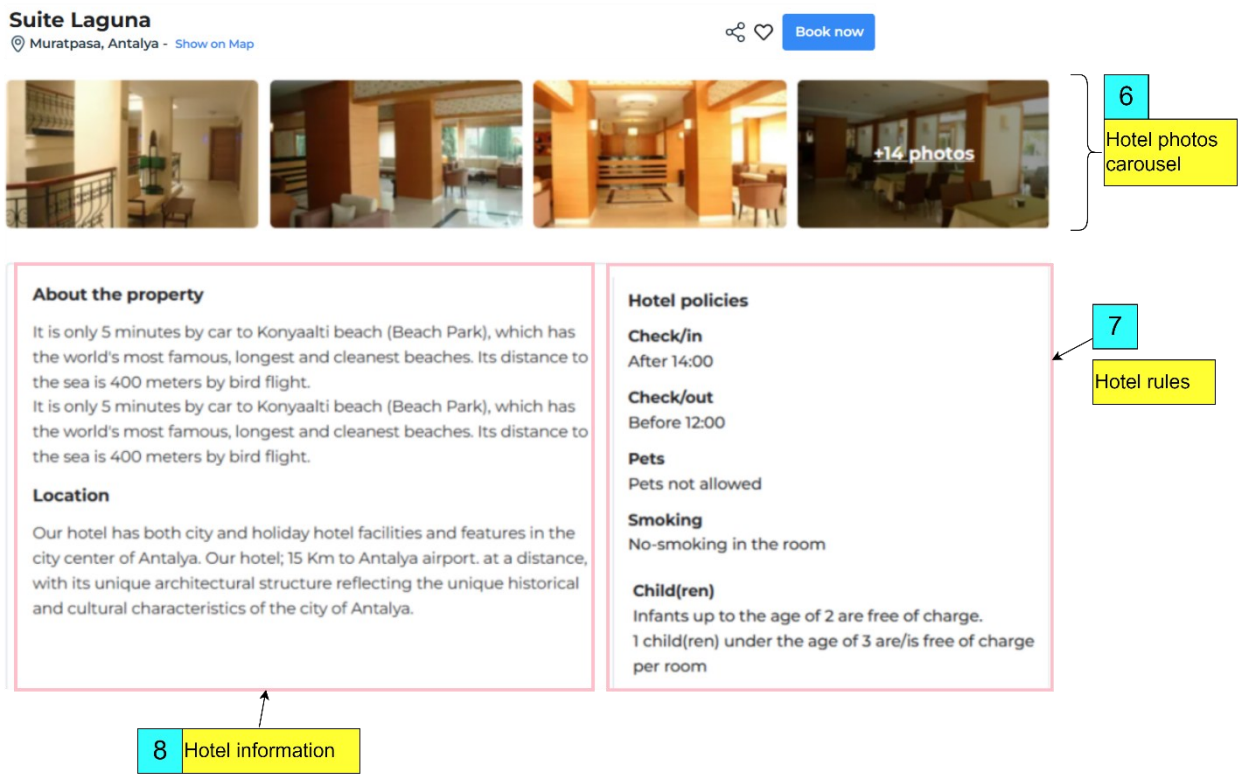


Figure 4: when user opens the hotelDetails.html page first time

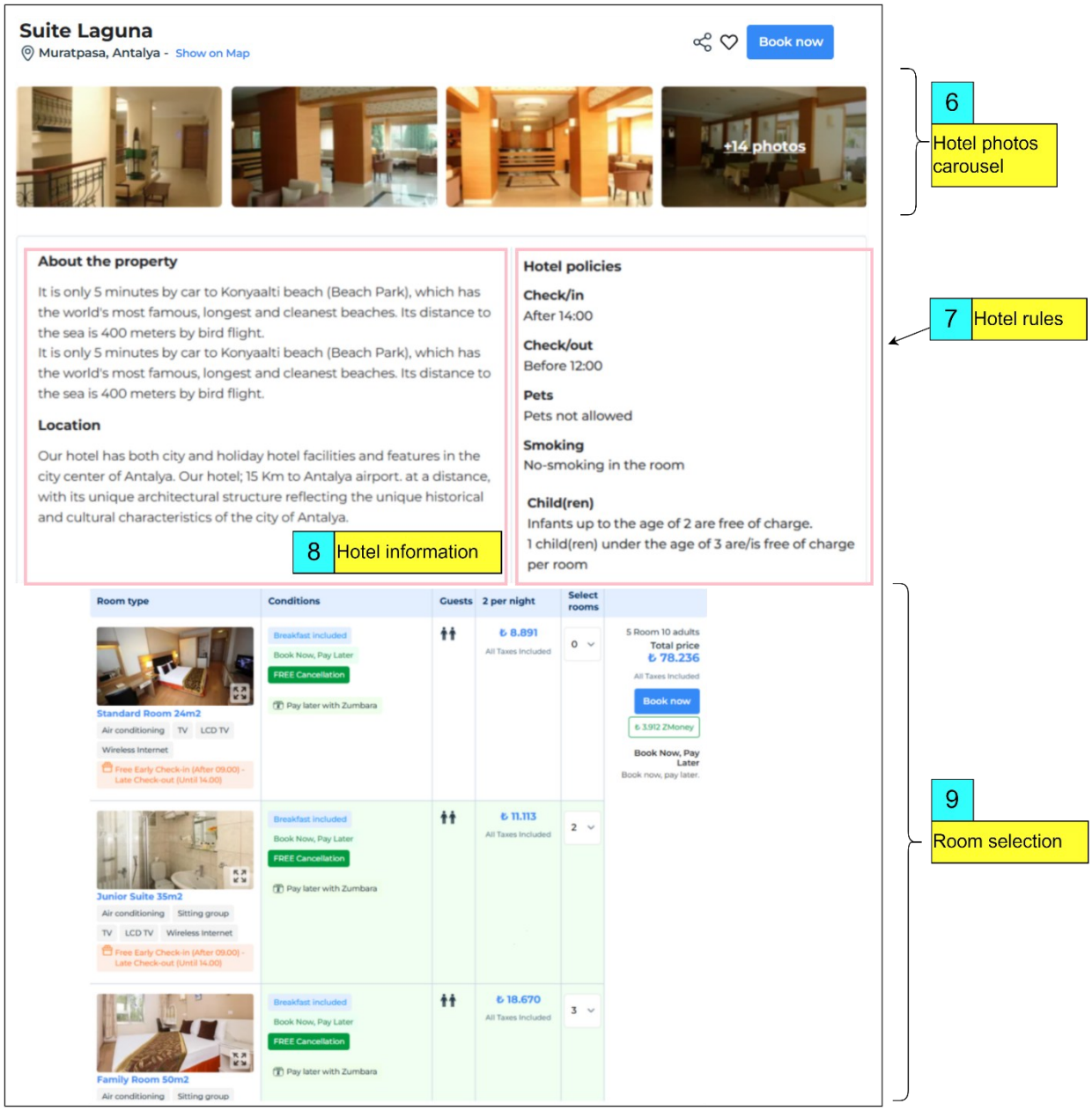


Figure 5: when user clicks “Book now” button

d. payment.html

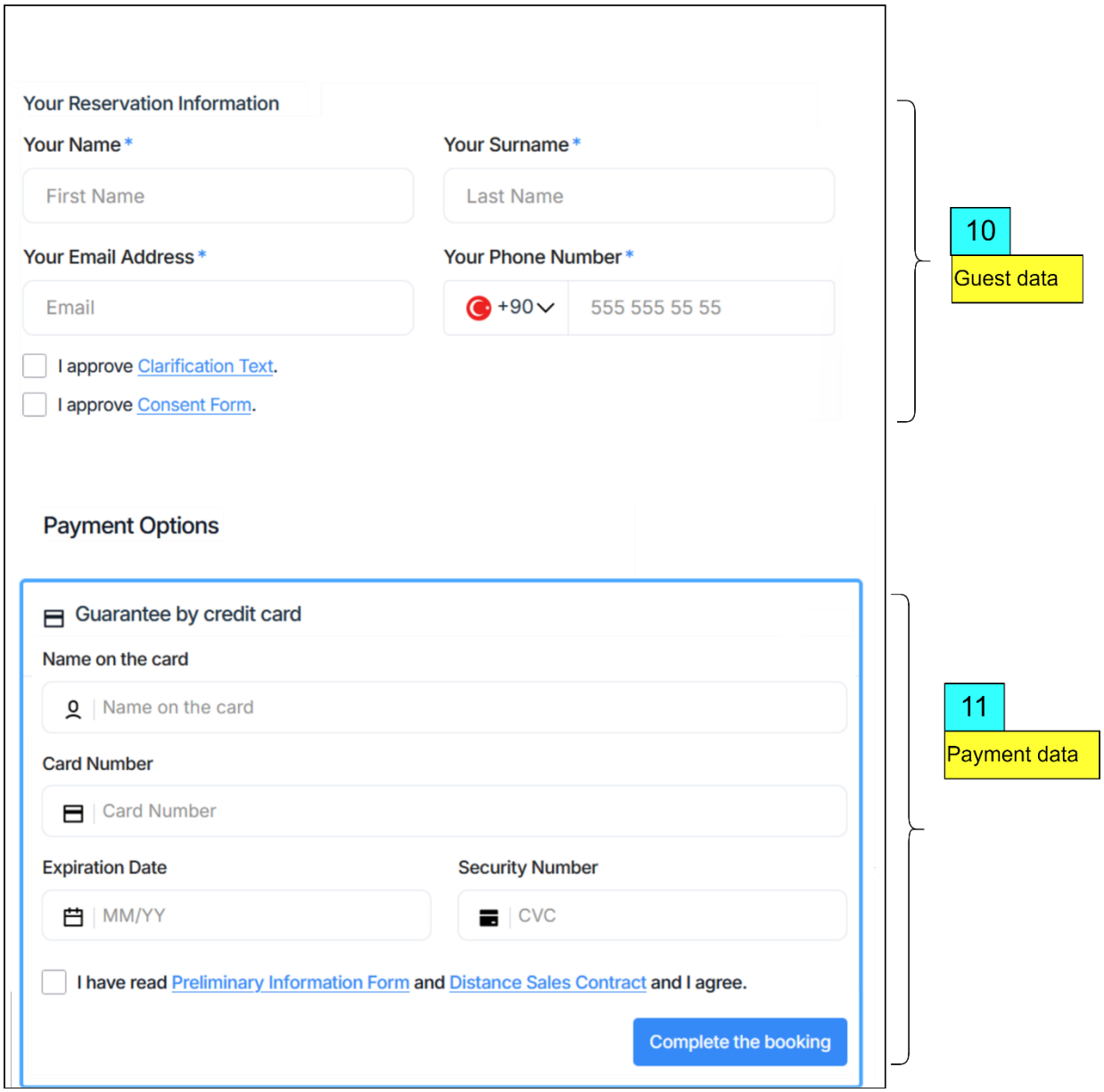


Figure 6: Payment.html

9. RUBRIC

- Groups (or individuals) **may be** subjected to interviews to demonstrate mastery of the subject. Every member must be present at the project demo. Otherwise he/she will get 0.
- You must upload a working project. It is not my responsibility to debug and fix your application. Develop your app step by step. Once you are sure of a step, continue developing the next step.
- Don't forget the rule from Syllabus: In order to pass this course, each project's grade must be at least 30/100
- Copying the work of others is not permitted.
- Projects must be uploaded to the LMS. Late submissions or submissions via email will not be accepted. There will be no makeup for a missing project.
- If you develop the project in a group, don't forget that being involved in a project and working in harmony with the team is very important. Every member should be given equal work. Create a task list for the application and divide tasks among the team members. During the demonstration, I may ask you which part of the project was developed by which student. Every member must know the general concept and rules of the project. I may ask random questions to each member.

PART-1: Penalties	**Points**
Using an unapproved technology (Dom processing tools, frameworks, or libraries such as JQuery, ReactJS, Node, Express, Templates, Easy-to-Use Drag-and-Drop Builders, etc)	[-100,-10]
Using "inline" or "embedded" style CSS or JavaScript	[-50,-1]
Late submission or not submitting project	Fail reason

index.html	**40**
searchBar	5
deals and discounts section	5
popular searches section and displaying results	10
search carousel	5
search function	10
responsive design	5
searchResults.html	**20**
search function	5
responsive design	15
hotelDetails.html	**20**
hotel details section	5
rooms section and functionalities	10
responsive design	5
payment.html	**20**
forms	10
responsive design	10
SUM	100